

Apunte 4

Priscilla Romero Barquero - 2023332718

Organización de los datos en archivos de datos

En el almacenamiento de archivos, puedo tener registros de tamaño fijo, es decir, todos los registros ocupan siempre el mismo espacio.

Esto facilita estimar cuántos registros contiene un archivo, además de mantenerlo siempre lleno y evitar que se fragmente.

En un archivo básico de base de datos, se incluye un header de 24 bytes, que sirve para saber cómo se organizan los datos.

Si realizo borrados, también debo llevar un puntero hacia la lista de registros borrados, la cual forma una lista simple.

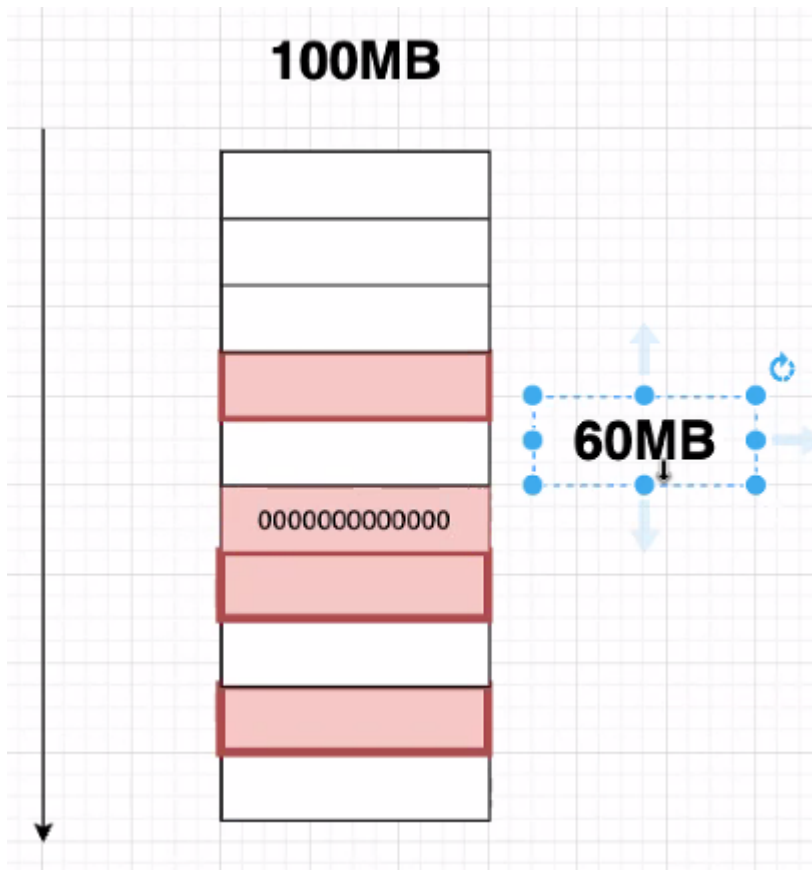
De esta forma, el algoritmo de escritura puede detectar si existen elementos borrados y, si no los hay, escribir directamente al final del archivo.

	24	394	4	98	320	
header	head 4 bytes	EOF 4 bytes	# records 4 bytes	free 4 bytes	tail 4 bytes	4
24	1					246
98						-1
172	5					320
246	4					172
320	7					-1
394						

Imaginemos un archivo que mide 100 megas y contiene aproximadamente 10 registros. Cuando se agregan datos, estos se van insertando dentro del archivo. Supongamos ahora que algunos registros, representados en rojo, han sido borrados.

Desde el punto de vista del sistema de archivos, el archivo sigue pesando 100 megas, porque su organización interna está basada en inodes.

Sin embargo, de manera lógica, el archivo pesa solo 70 megas, ya que hay 7 registros activos y 3 borrados. Es muy común que los registros se borren de forma lógica, es decir, que sigan existiendo dentro de la base de datos, pero lo que se elimina es el puntero que hacía referencia a ellos.

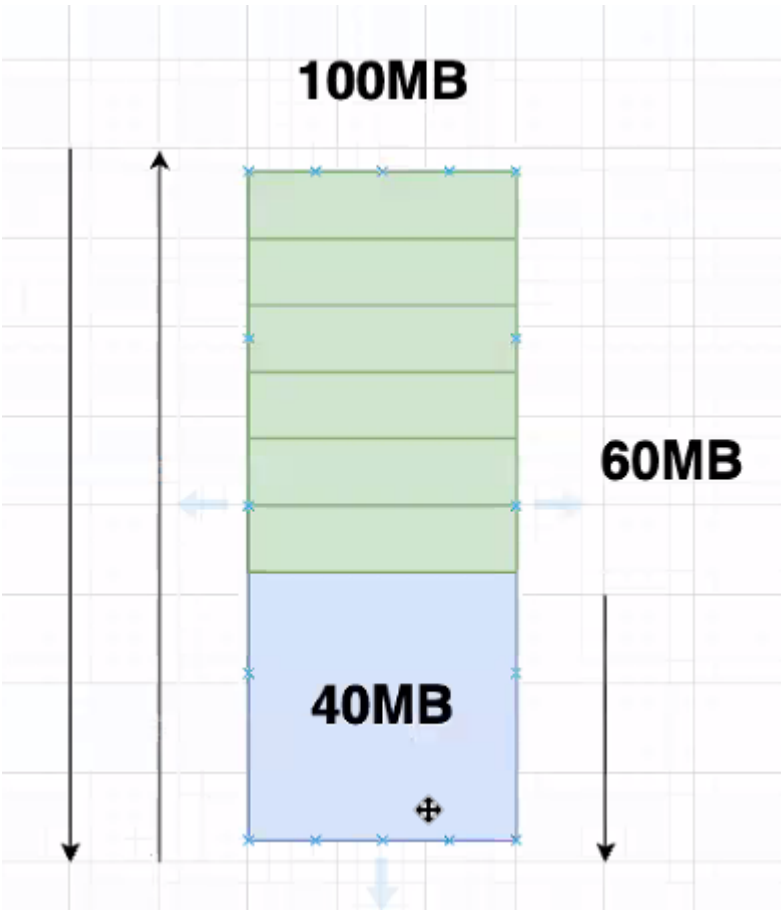


Hace algún tiempo se realizaba la defragmentación: este proceso consistía en recorrer los datos, verificar cuáles estaban correctos y, al encontrar un dato borrado, copiar el siguiente registro disponible hacia esa ubicación, sobrescribiéndolo.

Esto provocaba que al final del archivo quedaran megas disponibles correspondientes a los datos borrados, pero ese espacio se mantenía reservado para poder ser reutilizado más adelante.

En estos casos, esos megas finalmente se devuelven al sistema operativo para que pueda usarlos libremente. Una ventaja de trabajar con registros de tamaño fijo es que no causan fragmentación de archivos, ya que todos ocupan el mismo espacio.

En cuanto al borrado a bajo nivel, si necesito eliminar un registro de forma segura, además de quitar el puntero, se sobrescriben todos sus bits con 000, asegurando que los datos no puedan recuperarse.



Archivos variables

Que pasa cuando nos cambiamos de bando? El cambio de bando es cuando nos movemos de un registro de tamaño fijo a uno variable.

El archivo de datos tiene header, el tamaño del registro y otras variables tiene esta forma:

Suponga que se agrega un registro con los siguientes datos:

39, Nereo, Campos, donde se incluyen la edad, el nombre y el apellido.

size 2 bytes	id int = 4bytes	age short=2 bytes	size 1 bytes	name max=32	size 1 bytes	last_name max=32
-----------------	--------------------	----------------------	-----------------	----------------	-----------------	---------------------

El proceso consiste en guardar el registro armándolo desde el final hacia el inicio.

Primero se guarda el apellido, que en este caso es "Campos".

Luego se guarda su tamaño, que es 6.

Después se almacena el nombre, "Nereo", junto con su tamaño, que es 5.

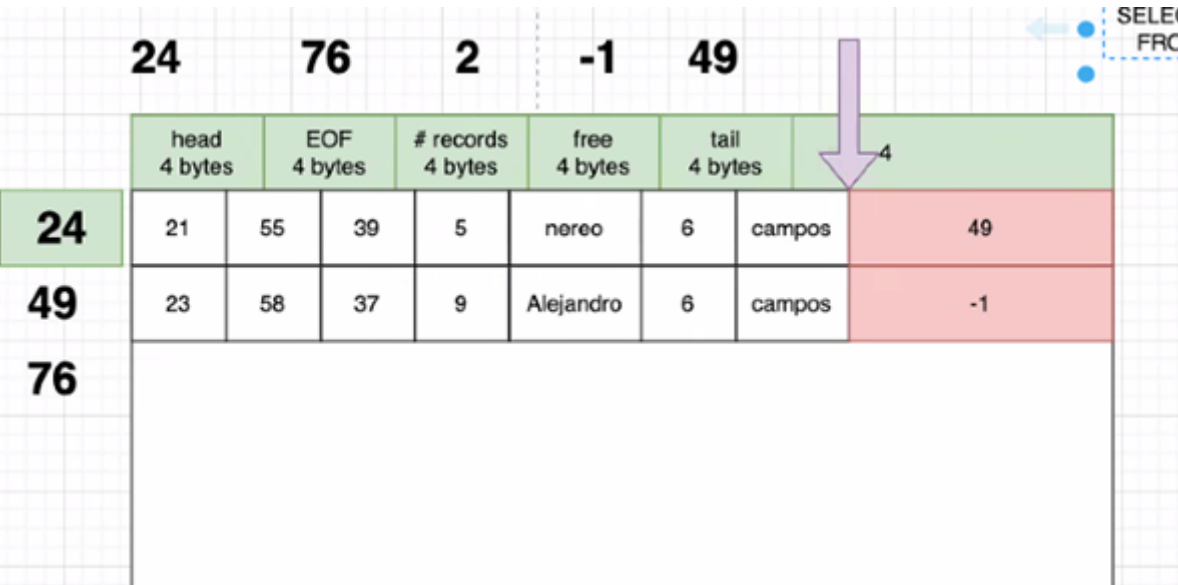
A continuación se guarda la edad, que ocupa 2 bytes.

Finalmente se guarda el identificador (ID) y el tamaño total del registro, que se calcula al final del proceso.

2	4	2	1	5	1	6
21	55	39	5	nereo	6	campos

Al final, el registro mediría 21 bytes y tendría un puntero que apunta al siguiente registro con el valor -1. Después de eso, el dato se inserta en el archivo. El primer valor siempre coloca el puntero en -1.

Si se desea insertar un nuevo registro, este se ubicará en el espacio que corresponda según su tamaño. Como el tamaño de los registros es variable, esto complica las operaciones de inserción, borrado y actualización.



Si se realiza una consulta para encontrar el nombre y apellido de las personas con edad 39, es necesario ir a buscar la información, extraerla y, con base en su tamaño, determinar en qué parte del archivo puede estar el registro.

Este método permite aprovechar mejor el espacio, evitando desperdicio. Sin embargo, presenta un problema de fragmentación de datos.

Por ejemplo, si en un archivo se borran ciertos registros con tamaños específicos y luego se desea insertar un nuevo registro que necesita más espacio del disponible de forma continua, no habrá lugar suficiente para colocarlo.

En ese caso, será necesario solicitar más recursos al sistema operativo.

NOTA

En una base de datos siempre se intenta evitar almacenar datos binarios. Esto significa que no se deben guardar directamente audios, imágenes o videos, especialmente en bases de datos SQL. Este tipo de datos provoca fragmentación dentro de la base de datos y, además, no aporta valor desde el punto de vista de las búsquedas. También se recomienda evitar los datos de tamaño variable. Si se respetan estas reglas, los registros siempre tendrán un tamaño fijo y no variable.

Búsqueda en listas

En una lista, la búsqueda se realiza de forma simple. Para encontrar un registro entre n registros, se necesitan n comparaciones. Esto sucede porque en una base de datos, cuando se usa una lista simple, el algoritmo debe recorrer los registros uno por uno.

Existen dos casos extremos:

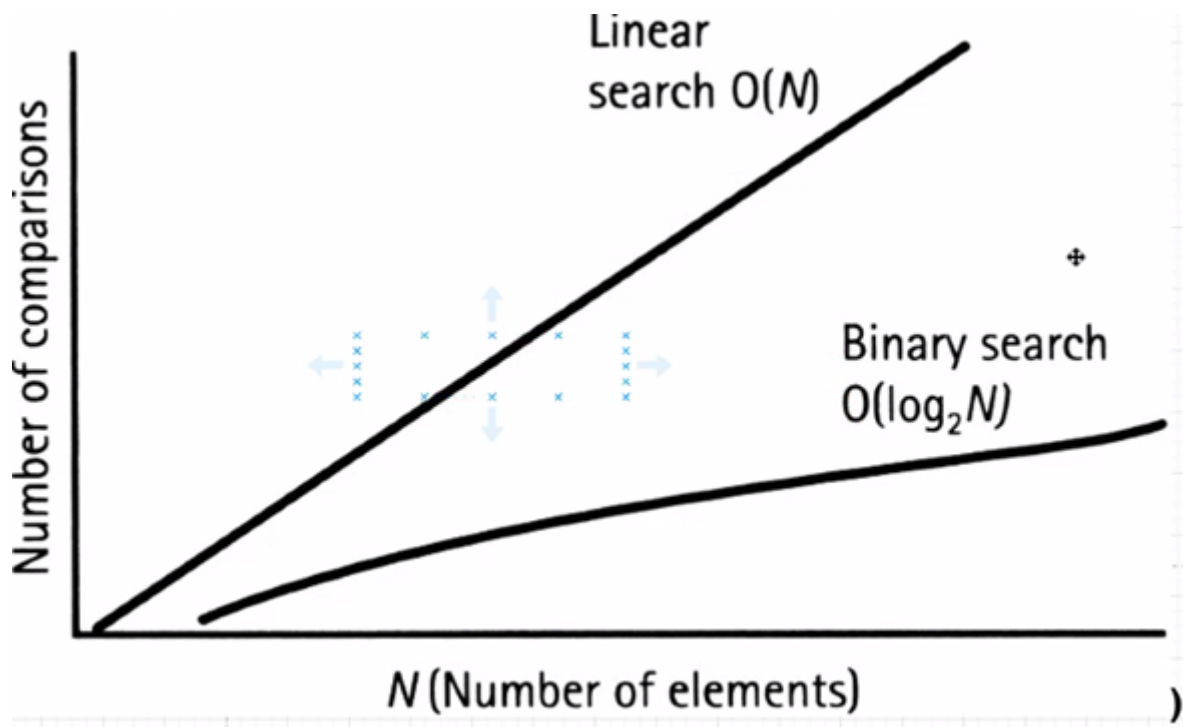
1. Que el registro no se encuentre en la lista.
2. Que el registro se encuentre hasta el final de la lista.

Los algoritmos tienen una notación llamada Big O, que indica cómo crece su complejidad.

En este caso, el crecimiento es lineal, lo que significa que en el peor de los casos se realizan n comparaciones para determinar si un dato está o no.

Cuando se insertan registros al final o de forma ordenada, también puede ser necesario llegar hasta el final para confirmar si el dato existe o no.

Una alternativa sería usar una búsqueda binaria, que es mucho más rápida, ya que conforme crece la cantidad de datos, el tiempo de respuesta aumenta más lentamente.



El método de búsqueda binaria es útil, pero solo tiene sentido cuando los datos se manejan en memoria. Esto se debe a que en memoria sabemos que los elementos están ordenados y conocemos cuántos hay, por lo que el algoritmo puede ir recortando la lista a la mitad en cada paso.

El problema con las bases de datos es que no siempre se sabe si el tamaño del archivo está directamente relacionado con la cantidad de registros.

Además, podría ocurrir que se intente acceder a un registro que ya fue borrado.

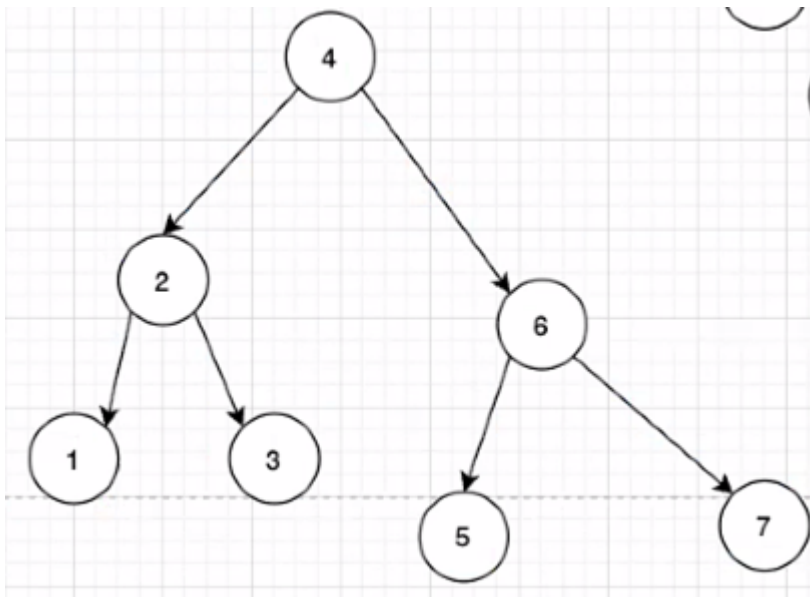
Por esa razón, la búsqueda en una base de datos normalmente es lineal, también conocida como un SCAN. Esto significa que hay que revisar todos los datos uno por uno. Sin embargo, existen estructuras más eficientes para recorrer y buscar datos dentro de una base de datos, como los árboles binarios de búsqueda.

Aun así, los árboles binarios por sí solos no siempre son la mejor opción.

Si los datos se insertan en orden, todos terminan cayendo en el hijo derecho, haciendo que el árbol se doble completamente y se convierta en una lista simple.

En ese caso, la búsqueda vuelve a ser lineal, igual que un SCAN.

Por eso se necesitan estructuras **balanceadas**, que eviten que el árbol se vuelva demasiado profundo y en su lugar se ensanche, facilitando una búsqueda más rápida y eficiente.



Índices y eficiencia en las búsquedas

Usar estructuras balanceadas permite hacer menos comparaciones al buscar un dato específico.

Si aplicamos esta idea a la estructura de un archivo de base de datos, podríamos organizarlo siguiendo el principio de un árbol balanceado.

Esto implica gastar un poco más de almacenamiento, pero a cambio se obtienen búsquedas mucho más rápidas.

Aquí es donde entran los tipos de índices.

Existen dos tipos principales, que son estructuras que acompañan a los datos y sirven para aumentar la velocidad de búsqueda.

